Recitation 11 – Network Layer

Daniel Karalnik and Liam Tal

Computer Networks

Table of Contents

1. IP Addressing
2. Forwarding vs Routing
3. IPv4 Datagram Format
4. IP Fragmentation
5. NAT (Network Address Translation)

Today

- Network Layer
  - IP Addressing
  - Forwarding
  - IP Datagram
  - IP Fragmentation
  - NAT

IP Addressing

IPv4 Address

An **IP(v4) address** is 4 bytes (32 bits) long.

How many different IPv4 addresses exist? $2^{32} \approx 4.3$ billion

Notation: Usually written as `BYTE.BYTE.BYTE.BYTE`

- Example: `193.32.216.9`
- Binary: `11000001 00100000 11011000 00001001`

Subnets

Key Concepts

- **Subnet:** A logical subdivision of an IP network where hosts can communicate **directly without a router**
- **Subnet mask:** Defines which bits are the **network prefix** vs. the **host identifier**

Subnet Notation

CIDR Notation

Example: `223.1.1.0/24`

- /24 means first 24 bits are network
- Remaining 8 bits for hosts
- Represents: `223.1.1.xx`

Netmask Notation

Example: `255.255.255.0`

- AND with IP address
- `223.1.1.0` netmask `255.255.255.0`
- Result: `223.1.1.xx`

Exercise 1

Question

A network on the Internet has a subnet mask of **255.255.240.0**
What is the **maximum number of hosts** it can handle?

Exercise 1

Question

A network on the Internet has a subnet mask of **255.255.240.0**
What is the **maximum number of hosts** it can handle?

Solution:

`255.255.240.0 = 11111111 11111111 11110000 00000000`

- Network bits: 20 (the 1s)
- Host bits: 12 (the 0s)

Maximum addresses: $2^{12} = \mathbf{4096}$
(Usable hosts: $2^{12} - 2 = 4094$, since network address and broadcast are reserved)

Exercise 2

Question

Suppose an ISP owns the block of addresses **111.37.192.0/20**.
Suppose all customers have networks of size **400 addresses** each.
For how many customers can the ISP allocate address blocks?

1. 8
2. 9
3. 10
4. 11
5. None of the above

Exercise 2 - Solution

Step 1: How many addresses in `111.37.192.0/20`?

`01101111 00100101 1100XXXX XXXXXXXX`
$2^{12} = 4096$ addresses

Step 2: Minimal block size with > 400 addresses?

- $2^8 = 256$ (too small)
- $2^9 = 512$ (sufficient!) $\rightarrow$ /23 block

Step 3: How many /23 blocks fit in a /20 block?
$2^{23-20} = 2^3 = \mathbf{8}$

Answer: 1. (8 customers)

Forwarding vs Routing

Forwarding

Moving a packet from the router's **input link** to an **output link**

- Local action
- Uses forwarding table
- Hardware-based (fast)

Routing

Determine the **route or path** taken by packets from sender to receiver

- Global action
- Uses routing protocols
- Software-based

Longest-Prefix Matching: When multiple routing entries match a destination, use the one with the **most specific (longest) prefix**.

Exercise 3

Routing Table:

Address prefix	Next hop
196.94.2.0/24	A
196.94.2.128/25	B
196.94.0.0/16	C
196.94.64.0/18	D
196.76.0.0/14	E
140.0.0.0/8	F
128.0.0.0/2	G
96.0.0.0/2	H
0.0.0.0/0	I

Find next hop for:

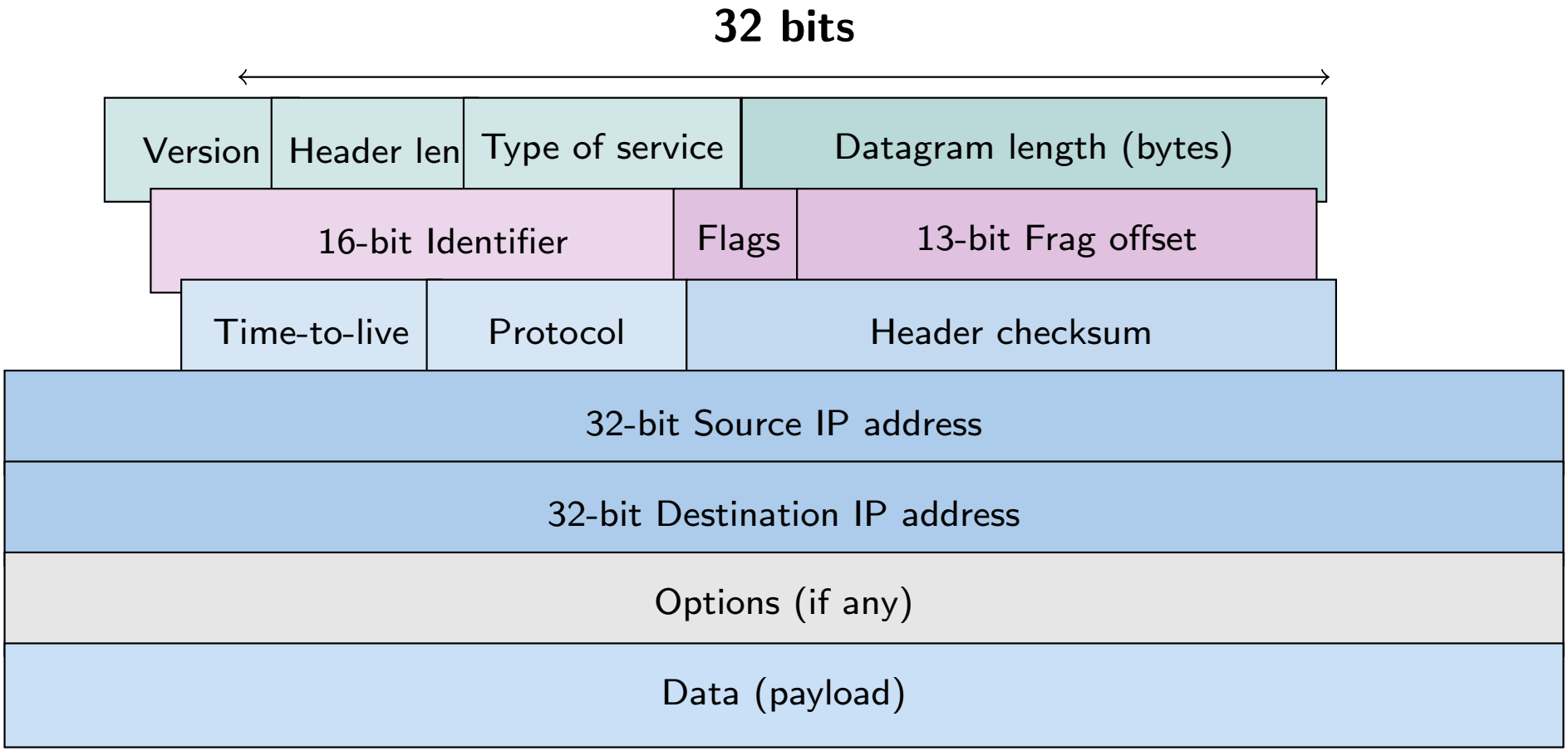
#	Destination IP
1	139.1.1.1
2	196.94.2.100
3	196.94.2.200
4	196.94.3.100
5	32.2.3.4
6	64.94.2.0

Exercise 3 - Solution

#	Destination IP	Matching Prefix	Next Hop
1	139.1.1.1	128.0.0.0/2	**G**
2	196.94.2.100	196.94.2.0/24	**A**
3	196.94.2.200	196.94.2.128/25	**B**
4	196.94.3.100	196.94.0.0/16	**C**
5	32.2.3.4	0.0.0.0/0	**I**
6	64.94.2.0	0.0.0.0/0	**I**

Remember: Always use the **longest matching prefix**!
Example: 196.94.2.200 matches both /24 and /25, but /25 is longer $\rightarrow$ B

Recap: IPv4 Datagram Format



IPv4 Header – True/False Questions

Question 1

IP *Header length* field includes Transport layer (TCP/UDP) header length.

IPv4 Header – True/False Questions

Question 1

IP *Header length* field includes Transport layer (TCP/UDP) header length.

False

- Header length field specifies **only the IP header** size
- Field is in **4-byte (32-bit) words**: value of 5 $\rightarrow$ 20 bytes
- Minimum: 20 bytes (no options), Maximum: 60 bytes
- IPv6 simplified this to a fixed 40-byte header

IPv4 Header – True/False Questions

Question 2

Time-to-live is measured in milliseconds.

IPv4 Header – True/False Questions

Question 2

Time-to-live is measured in milliseconds.

False

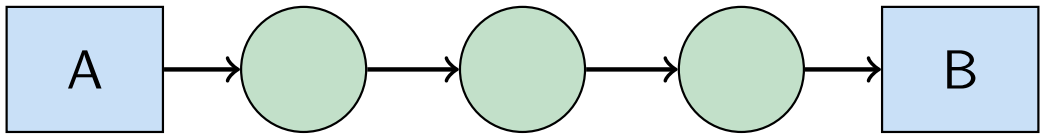
- It is measured in **hops**
- Each router decrements TTL by 1
- When TTL = 0, the packet is dropped
- In IPv6: renamed to "Hop Limit"

Why "time" to live? See RFC 791 – originally intended to be decremented every second, but in practice only decremented at each hop.

IPv4 Header – True/False Questions

Question 3

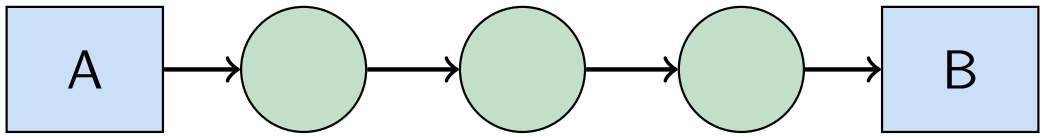
An IP datagram sent from A to B will have the same IP header checksum when sent from A and when arrived at B.



IPv4 Header – True/False Questions

Question 3

An IP datagram sent from A to B will have the same IP header checksum when sent from A and when arrived at B.



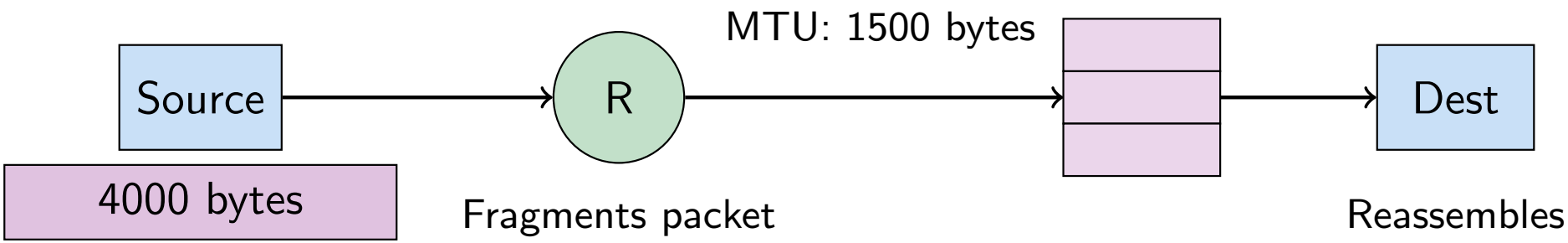
False

- TTL changes at each hop $\rightarrow$ checksum must be recalculated
- Checksum covers the entire IP header

Recap: IP Fragmentation – Why?

Why Fragmentation?

To send datagrams over links with smaller **MTU** (Maximum Transmission Unit)



Recap: IP Fragmentation – How?

Fragmentation Rules

- Fragment payload must be multiple of **8 bytes** (except last fragment)
- All fragments share the **same 16-bit ID** as original datagram
- **Flags** (3 bits): `0|DF|MF`
  - **DF** (Don't Fragment): if set, drop packet instead of fragmenting
  - **MF** (More Fragments): set to 1 on all fragments except the last
- **Offset**: byte position in original payload $\div$ 8

Recap: IP Fragmentation – why?

Where Fragment?

At any router where outgoing link MTU is smaller than the datagram

Where Reassemble?

Only at the **final destination**

IPv6: Fragmentation only at sender/receiver. Routers send ICMP "Packet Too Big" instead.

Exercise 4: Fragmentation – Fragmentation

Question

Consider sending a **3370-byte datagram** into a link with **MTU of 500 bytes**.
The original datagram has **identification number 780**.

1. How many fragments are generated?
2. What are their characteristics (ID, sizes, offsets, flags)?

Exercise 4: Fragmentation – Solution Step 1

Step 1: Calculate payload sizes

Original datagram: 3370 bytes

- IP header: 20 bytes
- Payload: $3370 - 20 = \mathbf{3350\ bytes}$

Each fragment (MTU = 500 bytes):

- IP header: 20 bytes
- Max payload: $500 - 20 = 480$ bytes
- Must be multiple of 8: $\lfloor 480/8 \rfloor \times 8 = \mathbf{480\ bytes}$

Step 2: Number of fragments

$\lceil 3350/480 \rceil = \lceil 6.98 \rceil = \mathbf{7\ fragments}$

Exercise 4: Fragmentation – Solution Step 2

Fragment Table:

#	ID	Data Size	Offset	MF
1	780	480	0	1
2	780	480	60 (480/8)	1
3	780	480	120 (960/8)	1
4	780	480	180 (1440/8)	1
5	780	480	240 (1920/8)	1
6	780	480	300 (2400/8)	1
7	780	470	360 (2880/8)	0

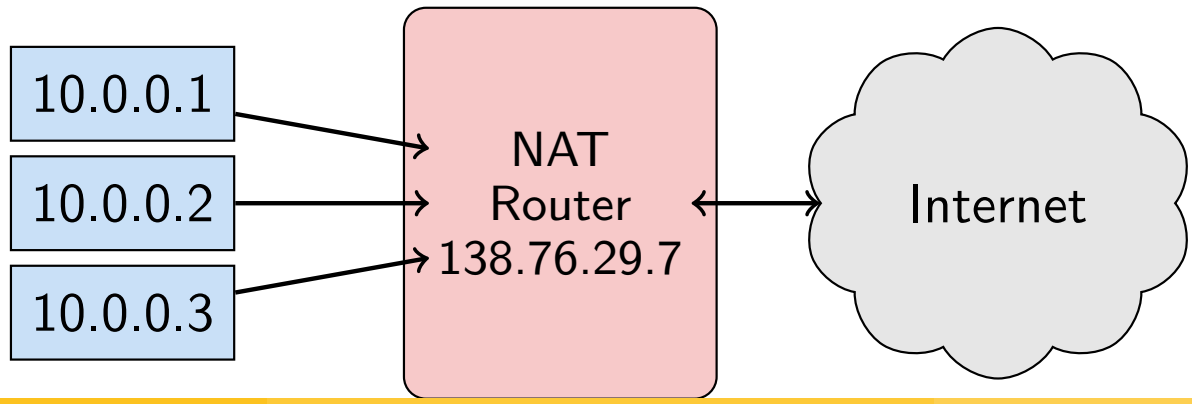
Note: Last fragment has 470 bytes ($3350 - 6 \times 480 = 470$) and MF $= 0$ (no more fragments)

Intro to NAT

Network Address Translation

Translates **private IP addresses** to **public IP addresses** (and vice versa) by rewriting packet headers.

- Maintains a **translation table**: maps (external IP, external port) $\leftrightarrow$ (internal IP, internal port)
- Allows multiple internal hosts to share **one public IP** (using different ports)
- Port field is 16 bits $\rightarrow$ up to 65,535 simultaneous connections



Why NAT?

Motivation: One IP for the Outside World

- **No need for ISP allocation:** One public IP address is used for all internal devices
- **Internal flexibility:** Can change addresses of devices in local network without notifying outside world
- **ISP independence:** Can change ISP without changing addresses of internal devices
- **Security benefit:** Devices inside local network are not explicitly addressable/visible by outside world

Capacity: 16-bit port field $\rightarrow$ 60,000+ simultaneous connections with a single LAN-side address!

NAT Controversy

Criticisms of NAT

- **Layer violation:** Routers should only process up to layer 3 (Network), but NAT inspects/modifies layer 4 (Transport) port numbers
- **Violates end-to-end argument:** Hosts should be able to communicate directly without intermediate modification
  - Application designers must account for NAT (e.g., P2P applications)
- **Address shortage:** Should be solved by IPv6, not NAT workarounds

Reality: Despite criticisms, NAT is ubiquitous due to IPv4 address exhaustion and practical benefits.

NAT Translation Table

WAN side	LAN side
138.76.29.7:5001	10.0.0.1:3345
138.76.29.7:5002	10.0.0.2:3346
138.76.29.7:5003	10.0.0.1:3347

NAT Traversal Problem

The Problem

How can **external hosts** initiate connections to **internal hosts**?

- Server address (e.g., `10.0.0.1`) is local to LAN – client can't use it as destination
- Only one externally visible NATted address (e.g., `138.76.29.7`)
- NAT table entries are created by *outgoing* packets, not incoming

Three main solutions:

1. Static port forwarding
2. UPnP (Universal Plug and Play)
3. Relaying

NAT Traversal: Solution 1 – Static Port Forwarding

Static Configuration

Manually configure NAT router to forward incoming connections at a given port to an internal server.

Example:

- External: `138.76.29.7:2500`
- Always forwarded to: `10.0.0.1:2500`

Pros

- Simple to set up
- Reliable
- No extra protocols

Cons

- Manual configuration
- Not scalable
- Requires router access

NAT Traversal: Solution 2 – UPnP IGD

Universal Plug and Play – Internet Gateway Device Protocol

Allows NATted hosts to **automatically** configure port mappings on the router.

Capabilities:

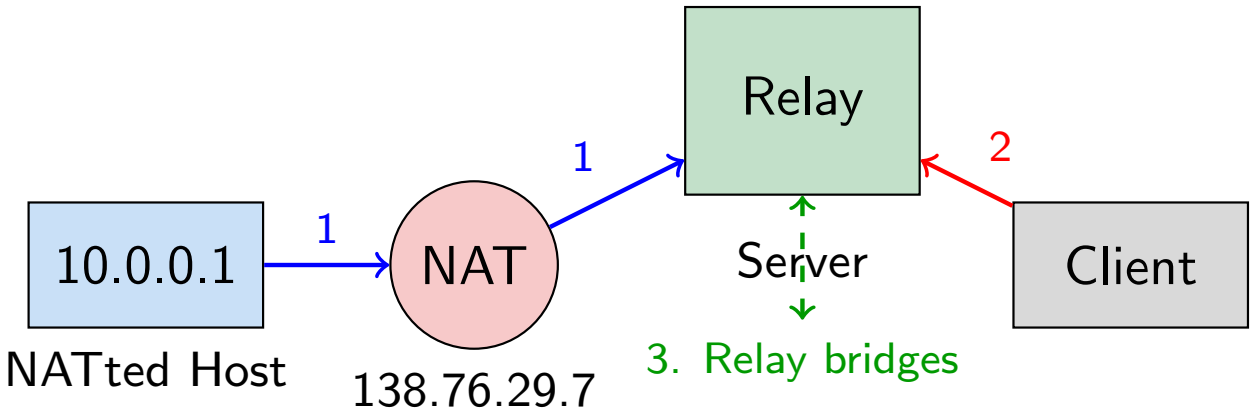
- **Learn public IP address** (e.g., discover that external IP is `138.76.29.7`)
- **Enumerate existing port mappings** on the router
- **Add/remove port mappings** (with lease times)

Use case: Gaming consoles, P2P applications automatically opening ports without manual router configuration.

NAT Traversal: Solution 3 – Relaying

Relay Server

Use a third-party server with a public IP to relay traffic between NATted hosts.

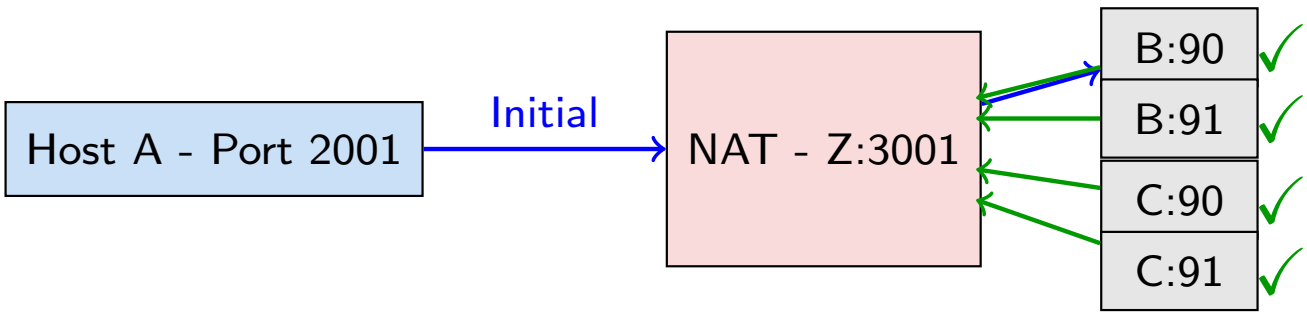


1. NATted host initiates connection to relay
2. External client connects to relay
3. Relay bridges packets between both connections

NAT Types: Full Cone

Full Cone NAT

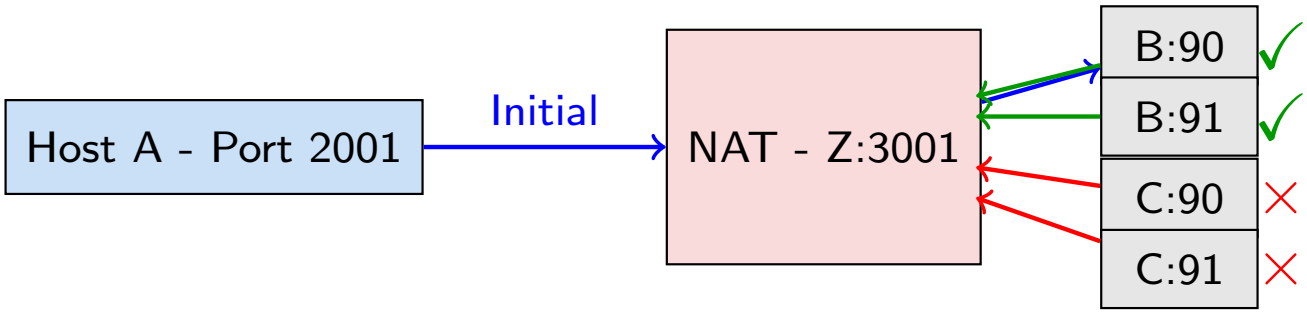
- Creates mapping between internal IP+Port and external IP+Port
- Once created, **any external host** can send to this external IP+Port



NAT Types: Address Restricted

Address Restricted Cone NAT

- Creates mapping between internal IP+Port and external IP+Port
- External host can send **only if** internal host previously sent to that **IP address**

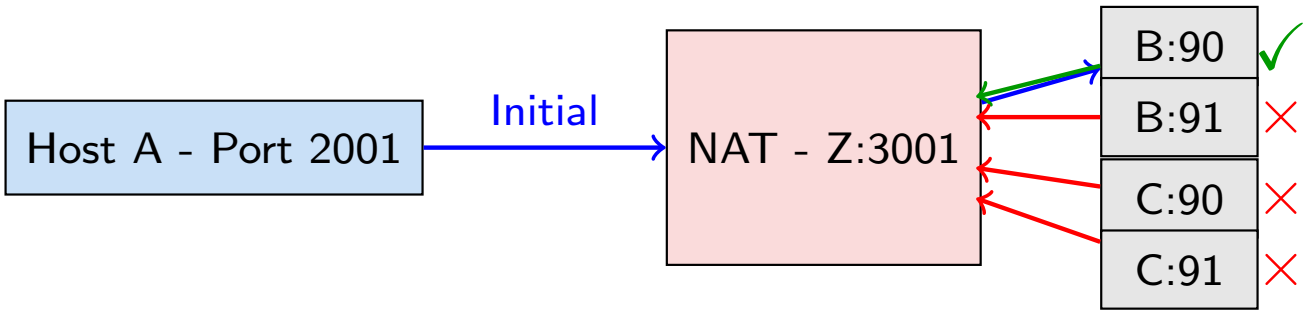


In this example we sent a packet to B:91, thus we may receive packets from any port with regard to host B.

NAT Types: Port Restricted

Port Restricted Cone NAT

- Creates mapping between internal IP+Port and external IP+Port
- External host can send **only if** internal host previously sent to that **IP+Port**

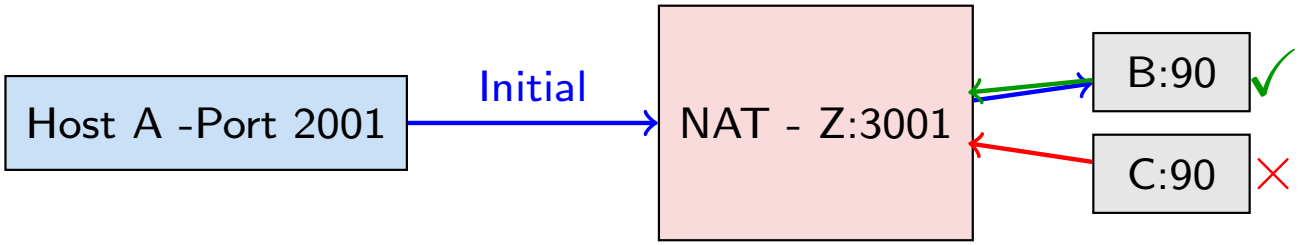


We are restricted by both **port and host** this time.

NAT Types: Symmetric

Symmetric NAT

- Creates mapping from internal IP+Port **AND destination IP+Port** to external IP+Port
- Different external port for each destination
- Most restrictive type



Note: Different destination → different external port allocated! while similar to the previous example, we will now also have multiple external mappings

NAT Types – Summary

NAT Type	Same ext port?	Who can reply?
Full Cone	Yes	Anyone
Address Restricted	Yes	Same IP (any port)
Port Restricted	Yes	Same IP+Port
Symmetric	No (per dest)	Same IP+Port

Restrictiveness (least to most):
Full Cone $<$ Address Restricted $<$ Port Restricted $<$ Symmetric

UDP Hole Punching – Overview

What is Hole Punching?

A technique that allows two hosts behind NATs to establish **direct peer-to-peer** communication without relaying.

Key idea:

- Both peers send packets to each other's NAT external address
- This creates NAT table entries ("punches holes")
- Once holes exist, direct communication is possible

Requires: A public server to help peers discover each other's external addresses (signaling server).

Full-Cone NAT Traversal (UDP)

Step 1: Port Mapping

- Both NATted hosts send packets to a public server
- NAT creates mappings: internal IP:port $\leftrightarrow$ external IP:port

Step 2: Address Discovery

- Public server tells each host the other's external address
- Host A learns: "Peer B is at `135.6.7.8:3000`"
- Host B learns: "Peer A is at `123.4.5.6:2000`"

Step 3: Data Exchange

- Hosts can now communicate directly!
- Full-cone NAT allows **any** external host to send through the mapping

Simple case: No hole punching needed – just address discovery!

Port-Restricted Cone NAT Traversal (UDP)

Problem: Port-restricted NAT only allows replies from the *exact* IP:port the internal host sent to.

Step 1: Port Mapping (same as full-cone)

Step 2: Mutual Address Discovery

- Public server tells each host the other's external address

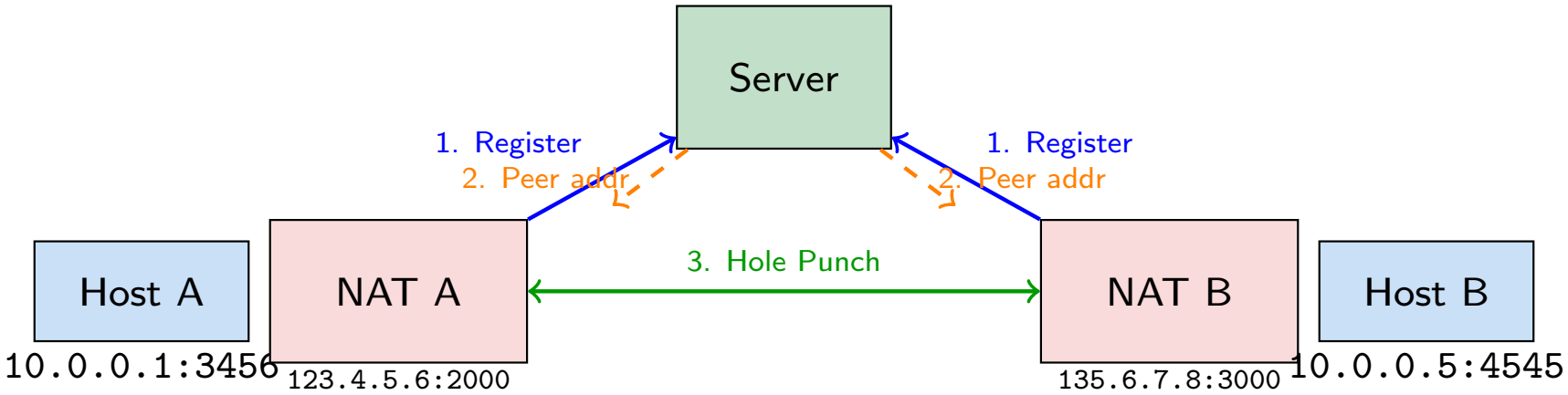
Step 3: Hole Punching

- Host A sends packet to Host B's external address
- This "punches a hole" in A's NAT for B's address
- Host B sends packet to Host A's external address
- This "punches a hole" in B's NAT for A's address
- **Now both NATs allow traffic from each other!**

Step 4: Data Exchange

- Direct peer-to-peer communication established

Hole Punching – Visualization



After hole punching: Direct P2P!

Limitation: Hole punching does **not work** with Symmetric NAT (different external port per destination).

Hole Punching – Summary

NAT Type	Hole Punching?	Notes
Full Cone	Not needed	Any host can send
Address Restricted	Works	Need to send first
Port Restricted	Works	Both must punch
Symmetric	**Fails**	Port changes per dest

Symmetric NAT Problem

With Symmetric NAT, when Host A sends to the server, it gets external port X. But when A sends to Host B, it gets a **different** external port Y. Host B doesn't know port Y!

Fallback for Symmetric NAT: Use relaying (TURN server in WebRTC).

Exercise 5 – NAT Types

Question

Alice and Bob are behind different NATs (UDP).

- Alice: internal `192.168.1.3:2000`, external router `56.67.78.89`
- Bob: internal `10.0.0.4:3000`, external router `12.23.34.45`

Bob sends packet to `56.67.78.89:2000`, then Alice sends to `12.23.34.45:3000`.
Alice's packet will arrive to Bob if Bob's NAT is:

1. Address-restricted-cone, but not port-restricted-cone
2. Port-restricted-cone, but not address-restricted-cone
3. Address-restricted-cone, and also port-restricted-cone
4. All of the above
5. **None of the above**

Exercise 5 – Solution

Step-by-step analysis:

1. Bob (internal `10.0.0.4:3000`) sends to `56.67.78.89:2000`
2. Bob's NAT creates mapping: internal $\rightarrow$ external port (e.g., `12.23.34.45:`**X**)
3. Alice sends to `12.23.34.45:`**3000**

Exercise 5 – Solution

Key Issue

Alice sends to port **3000** (Bob's *internal* port), but Bob's NAT likely assigned a **different external port** ($X \neq 3000$).
$\rightarrow$ The packet won't reach Bob's NAT mapping at all!

Answer: 5. None of the above
The NAT type is irrelevant – Alice is sending to the wrong port. NAT external ports are typically **not** the same as internal ports.